

Plan de Branching — Git Flow

Proyecto Sanos y Salvos — Evaluación Parcial N°2

DSY1106 Desarrollo Fullstack III

Integrante	Rol
Benjamin Arellano Gallardo	Desarrollador Fullstack
Benjamin Valdebenito	Desarrollador Fullstack
Maximiliano Vera	Desarrollador Fullstack
Matias Guzman	Desarrollador Fullstack

1. Introducción

El presente documento describe la estrategia de branching adoptada por el equipo para el desarrollo del proyecto Sanos y Salvos. La estrategia está basada en Git Flow adaptado, diseñada para facilitar el trabajo colaborativo, mantener un historial de cambios claro y garantizar la estabilidad del código en producción.

2. Estructura de Ramas

2.1 Ramas principales

main: Rama de producción estable. Solo recibe merges desde develop cuando el equipo valida que el estado del proyecto es entregable.

develop: Rama de integración continua. Todas las ramas feature hacen merge aquí una vez completadas.

2.2 Ramas de funcionalidad (feature)

Cada componente del sistema se desarrolló en una rama independiente con el prefijo **feature/** seguido del nombre del componente.

Rama	Descripción	Merge hacia
feature/msvc-mascotas	Microservicio mascotas — patrón Builder	develop
feature/msvc-reportes	Microservicio reportes — patrón Facade	develop
feature/msvc-usuarios	Microservicio usuarios — patrón Adapter	develop
feature/bff	Backend For Frontend — patrón Facade	develop
feature/infrastructure	API Gateway, Eureka, Config Server, etc.	develop
feature/frontend	Aplicación React con Vite y Keycloak	develop
feature/arquetipos	Arquetipos Maven reutilizables	develop
feature/docs	Documentación PDF del proyecto	develop

3. Flujo de Trabajo

El flujo seguido para cada componente fue:

1. Crear la rama feature desde develop:

```
git checkout develop
git checkout -b feature/nombre-componente
```

2. Desarrollar e implementar el componente.

3. Commit con mensaje descriptivo:

```
git add .
git commit -m "feat: descripcion del componente"
```

4. Push al repositorio remoto:

```
git push -u origin feature/nombre-componente
```

5. Merge a develop conservando la rama como evidencia:

```
git checkout develop
git merge feature/nombre-componente
git push origin develop
```

4. Gestión de Conflictos

4.1 Conflicto resuelto — Convención de nombres de ramas

Durante las etapas iniciales del proyecto, el equipo debatió sobre la convención de nombres para las ramas de trabajo. La propuesta inicial fue nombrar las ramas con el nombre del integrante responsable (por ejemplo, feature/benjamin o feature/matias).

Sin embargo, el equipo identificó que esta convención generaba ambigüedad: al revisar el historial de ramas era imposible saber qué funcionalidad se había desarrollado sin revisar el contenido del código, dificultando la trazabilidad del trabajo.

Resolución:

Se acordó por consenso adoptar la convención feature/nombre-componente, nombrando cada rama según el componente implementado. Esta decisión garantizó consistencia en todo el historial del proyecto.

Beneficios obtenidos:

- El historial de ramas refleja directamente los componentes desarrollados.
- Cualquier integrante puede identificar el estado del proyecto con un simple git branch.
- La trazabilidad entre commits, ramas y componentes es inmediata.

4.2 Conflicto resuelto — Push rechazado en main

Al intentar el primer push a main, el repositorio remoto rechazó los cambios con el error non-fast-forward, ya que GitHub había creado automáticamente un commit inicial (README) que no existía en el repositorio local.

Resolución:

Se utilizó `git push origin main --force` para sobrescribir el estado remoto, dado que el repositorio estaba en su etapa inicial y el commit remoto no contenía trabajo relevante. Para situaciones futuras en producción, se acordó usar siempre `git pull` antes de push.

5. Evidencia de Merges

Todas las ramas feature se mantienen sin eliminar en el repositorio remoto para evidenciar el proceso de desarrollo incremental. Los merges quedan documentados en el historial de commits de GitHub. El repositorio puede consultarse en:

<https://github.com/MafeXX/sanos-y-salvos-platform>